

JavaScript Complete Notes

1. Introduction to JavaScript

★ What is JavaScript?

JavaScript (JS) is a lightweight, interpreted or just-in-time compiled programming language with first-class functions. It is a prototype-based, multi-paradigm language.

JavaScript in One Line



JavaScript is the language of the web that adds interactivity, behavior and dynamic features to web pages.

★ Features of JavaScript

- Lightweight and interpreted language.
- Easy to learn and use.
- Full integration with HTML & CSS.
- Supports multiple programming paradigms (Object-Oriented, Functional, etc.).
- Event-driven and asynchronous.
- Cross-platform (Works on all modern browsers).
- Dynamic typing.
- Large standard library.
- Extensible and flexible.
- Used for both Frontend and Backend (Node.js).

★ History

JavaScript was created in 1995 by **Brendan Eich** while working at **Netscape Communications**. Initially it was called "**Mocha**", then "**LiveScript**" and finally named "**JavaScript**".



JavaScript

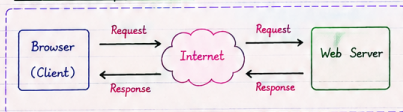
1995

Brendan Eich

★ Uses of JavaScript

- Making web pages interactive.
- Form validation.
- Creating animations and effects.
- Manipulating HTML and CSS.
- Handling events.
- AJAX (Asynchronous data handling).
- Creating Single Page Applications (SPA).
- Backend development using Node.js.

★ How JavaScript Works?



When a user requests a web page, the browser sends a request to the server. The server processes it and sends back the response. JavaScript makes this process dynamic and interactive.

★ JS Engine (V8)

JavaScript Engine is a program or an interpreter which executes JavaScript code. V8 is the JavaScript engine used in Google Chrome and Node.js. It converts JS code into machine code and executes it.



JavaScript Complete Notes

2. Variables & Data Types

★ Variables

Variables are containers used to store data values.

In JavaScript, we declare variables using `var`, `let` and `const`.

<u>var</u>	<u>let</u>	<u>const</u>
- Function scoped - Can be re-declared - Can be updated - Hoisted and initialized with undefined	- Block scoped - Cannot be re-declared - Can be updated - Hoisted but not initialized	- Block scoped - Cannot be re-declared - Cannot be updated - Hoisted but not initialized

Example

```

var x = 10;
let y = 20;
const z = 30;

x = 100; // allowed
y = 200; // allowed
z = 300; // not allowed

```

★ Naming Rules

- Must start with a letter, \$ or _
- Cannot start with a number
- Can contain letters, numbers, \$ and _
- Case-sensitive (`myVar` and `myvar` are different)
- Cannot use reserved keywords

Valid Examples

```

myVar
_name
$amount
user123

```

Invalid Examples

```

123var
my-var
let
const

```

★ Data Types

A. Primitive Data Types (Store single values)

①	String	Represents text.	Example: "Hello"
②	Number	Represents numbers.	Example: 25, 3.14
③	Boolean	Represents true or false.	Example: true, false
④	Null	Represents an intentional absence of value.	Example: null
⑤	Undefined	Variable declared but not assigned a value.	Example: undefined
⑥	Symbol	Unique and immutable value (ES6).	Example: Symbol('id')
⑦	BigInt	Used to store large integers (ES2020).	Example: 12345678901234567890n

B. Non-Primitive Data Types

(Store multiple values / collections)

- ★ **Object** {key : value}
Example: {name: "Z", age: 20}
- ★ **Array** [value1, value2, ...]
Example: [1, 2, 3]
- ★ **Function** function() { ... }
Example: function add() { }

★ typeof Operator

The `typeof` operator is used to find the type of a variable.

```

typeof "Hello" // string
typeof 25      // number
typeof true    // boolean
typeof null    // object (special case)
typeof undefined // undefined
typeof Symbol('id') // symbol
typeof 123n    // bigint
typeof [1,2,3] // object
typeof {a:1}   // object
typeof function(){} // function

```

Note :

- In JavaScript, arrays, objects and functions are treated as objects.
- null is of type "object" (this is a historical bug in JavaScript).

Quick Example

```

let a;
console.log(typeof a); // undefined
let b = null;
console.log(typeof b); // object
let c = [1,2,3];
console.log(typeof c); // object

```

JavaScript Complete Notes

3. Operators & Control Statements

★ Arithmetic Operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (Remainder)
**	Exponentiation
++	Increment
--	Decrement

Example

```
5 + 3 = 8
5 - 3 = 2
5 * 3 = 15
5 / 3 = 1.67
5 % 3 = 2
2 ** 3 = 8
let a = 5; a++; // 6
let b = 5; b--; // 4
```

Comparison Operators

==	Equal to
===	Strict equal to
!=	Not equal to
!==	Strict not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example

```
5 == '5' true
5 === '5' false
5 != 3 true
5 !== '5' true
5 > 3 true
5 < 3 false
5 >= 5 true
5 <= 4 false
```

★ Assignment Operators

=	Assign
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign
%=	Modulus and assign

Example

```
let x = 5;
x += 2; // x = 7
x -= 3; // x = 4
x *= 2; // x = 8
x /= 2; // x = 4
x %= 3; // x = 1
```

★ Logical Operators

&&	Logical AND
	Logical OR
!	Logical NOT

Example

```
(5 > 3) && (2 < 4) true
(5 > 3) || (2 > 4) true
!(5 > 3) false
```

★ Ternary Operator

condition ? expression_if_true : expression_if_false

Example :

let age = 18;

let result = (age >= 18) ? "Adult" : "Minor";

★ Control Statements

① if...else if...else

```
if (age >= 18) {
  console.log("Adult");
} else if (age > 13) {
  console.log("Teenager");
} else {
  console.log("Child");
}
```

② switch

```
let day = 3;
switch(day) {
  case 1: console.log("Mon"); break;
  case 2: console.log("Tue"); break;
  case 3: console.log("Wed"); break;
  default: console.log("Invalid");
}
```

When to use ?

- Use if...else when we have few conditions.
- Use switch when we have many fixed values to check.

★ Truthy & Falsy Values

Truthy Values (Evaluates to true)		Falsy Values (Evaluates to false)	
Value	Example	Value	Example
Non-empty strings	"hello", "0", "false"	false	false
Non-zero numbers	1, -1, 3.14	0	0, -0
Arrays (even empty)	[], [1, 2, 3]	Empty string	""
Objects (even empty)	{}, {a:1}	null	null
Function	function(){}	undefined	undefined
Infinity	Infinity	NaN	NaN
-Infinity	-Infinity		

Note :

In conditional statements, JavaScript converts the value to boolean. Understanding truthy & falsy values helps in writing clean code.



JavaScript Complete Notes

4. Loops & Functions

★ Loops

Loops are used to execute a block of code multiple times.

① For Loop

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

(init; condition; increment/decrement)

Output

1
2
3
4
5

④ Break Statement

Terminates the loop when a condition is met.

```
for (let i = 1; i <= 5; i++) {
  if (i == 3) break;
  console.log(i);
}
```

Output

1
2

② While Loop

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

Output

1
2
3
4
5

⑤ Continue Statement

Skips the current iteration and continues with the next iteration.

```
for (let i = 1; i <= 5; i++) {
  if (i == 3) continue;
  console.log(i);
}
```

Output

1
2
4
5

③ Do...While Loop

```
let i = 1;
do {
  console.log(i);
  i++;
} while (i <= 5);
```

Output

1
2
3
4
5

Note :

- Use **break** to exit the loop completely.
- Use **continue** to skip the current iteration.
- **for** loop is generally preferred when the number of iterations is known.

★ Functions

A function is a block of code designed to perform a particular task.

① Function Declaration

```
function greet(name) {
  console.log("Hello " + name);
}
```

greet("Z"); // Hello Z

② Function Expression

```
const greet = function(name) {
  console.log("Hello " + name);
};
```

greet("Z"); // Hello Z

③ Arrow Function

```
const greet = (name) => {
  console.log("Hello " + name);
};
```

greet("Z"); // Hello Z

(Single line)

```
const add = (a, b) => a + b;
```

④ Default Parameters

```
function multiply(a, b = 1) {
  return a * b;
}
```

console.log(multiply(5)); // 5
console.log(multiply(5, 3)); // 15

⑤ Rest Parameters

```
function sum(...numbers) {
  let total = 0;
  for (let num of numbers) {
    total += num;
  }
  return total;
}
```

console.log(sum(1, 2, 3, 4)); // 10

⑥ Return Statement

```
function square(n) {
  return n * n;
}
```

let result = square(4);
console.log(result); // 16

Important Points

- Functions help in code reusability.
- A function is executed when it is called.
- return statement sends a value back to the caller.
- If a function doesn't return anything, it returns **undefined**.

Function vs Procedure

Function	Procedure
Returns a value.	Does not return a value.
Used in expressions.	Used as a statement.
Example : let x = add(2, 3);	Example : printName();

Tips

- Use meaningful names.
- Keep functions small and focused.
- Use arrow functions for short and clean code.



JavaScript Complete Notes

5. Arrays & Objects

★ Arrays

An array is a special variable that can hold more than one value.

```
let fruits = ["Apple", "Banana", "Mango"];
           0       1       2
```

Common Array Methods

①	push()	Adds element at the end.	fruits.push("Orange"); // ["Apple", "Banana", "Mango", "Orange"]
②	pop()	Removes last element.	fruits.pop(); // ["Apple", "Banana"]
③	shift()	Removes first element.	fruits.shift(); // ["Banana", "Mango"]
④	unshift()	Adds element at the beginning.	fruits.unshift("Grapes"); // ["Grapes", "Apple", "Banana", "Mango"]
⑤	splice()	Adds/Removes elements.	fruits.splice(1, 1, "Pineapple"); // ["Apple", "Pineapple", "Mango"]
⑥	slice()	Returns a portion.	fruits.slice(1, 3); // ["Banana", "Mango"]

Other Useful Methods

- **concat()** - Merges arrays. `arr1.concat(arr2)`
- **indexOf()** - Returns index of element. `arr.indexOf("Banana")`
- **includes()** - Checks if element exists. `arr.includes("Mango")`
- **join()** - Joins all elements as string. `arr.join(", ")`
- **reverse()** - Reverses the array. `arr.reverse()`
- **sort()** - Sorts the array. `arr.sort()`
- **length** - Returns length. `arr.length`

Array Iteration

for loop	for (let i = 0; i < arr.length; i++) { console.log(arr[i]); }
for...of	for (let item of arr) { console.log(item); }
forEach()	arr.forEach(function(item) { console.log(item); });
map()	let newArr = arr.map(x => x * 2);
filter()	let even = arr.filter(x => x % 2 === 0);

Array Destructuring

```
let [a, b, c] = [10, 20, 30];
console.log(a, b, c); // 10 20 30
```

Spread Operator (...)

```
let arr1 = [1, 2, 3];
let arr2 = [...arr1, 4, 5];
console.log(arr2); // [1,2,3,4,5]
```

★ Objects

An object is a collection of key-value pairs.

```
let person = {
  name: "Zara",
  age: 20,
  city: "Delhi"
};
```

← key : value
← key : value
← key : value

Accessing Object Properties

Dot Notation	console.log(person.name); // Zara
Bracket Notation	console.log(person["age"]); // 20

Adding / Updating Properties

```
person.email = "zara@gmail.com"; // add
person.age = 21; // update
```

Deleting Properties

```
delete person.city; // removes city
```

Object Methods

```
let person = {
  name: "Zara",
  age: 20,
  greet: function() {
    console.log("Hello, I am " + this.name);
  }
};
person.greet(); // Hello, I am Zara
```

this keyword refers to the current object.

Object Destructuring

```
let { name, age } = person;
console.log(name); // Zara
console.log(age); // 20
```

Note :

- Arrays are ordered collections (index based).
- Objects are unordered collections (key based).
- Both arrays and objects are reference types.



JavaScript Complete Notes

6. Strings & DOM

★ Strings

A string is a sequence of characters used to represent text.

Creating Strings

```
let str1 = "Hello";
let str2 = 'JavaScript';
let str3 = `CodeWithZ`; // template literal (backticks)
```

Common String Properties & Methods

- length - Returns the length of the string.
- toUpperCase() - Converts to uppercase.
- toLowerCase() - Converts to lowercase.
- trim() - Removes whitespace from both ends.
- slice(start, end) - Extracts a part of string (end not included).
- substring(start, end) - Similar to slice().
- substr(start, length) - Extracts characters for given length.
- replace(old, new) - Replaces first occurrence.
- replaceAll(old, new) - Replaces all occurrences.
- includes(value) - Checks if value exists.
- indexOf(value) - Returns first index of value.
- lastIndexOf(value) - Returns last index of value.
- split(separator) - Splits to an array.
- charAt(index) - Returns character at index.
- concat(str2) - Joins two or more strings.
- startsWith(value) - Checks if string starts with value.
- endsWith(value) - Checks if string ends with value.
- repeat(n) - Repeats the string n times.

```
str.length; // 5
str.toUpperCase(); // HELLO
str.toLowerCase(); // hello
str.trim();
str.slice(0, 5); // Hello
str.substring(0, 5); // Hello
str.substr(0, 5); // Hello
str.replace("Hello", "Hi"); // Hi
str.replaceAll("l", "x"); // Hexxo
str.includes("lo"); // true
str.indexOf("o"); // 4
str.lastIndexOf("a"); // 7
str.split(" "); // ["Hello"]
str.charAt(1); // e
str.concat(" World"); // Hello World
str.startsWith("He"); // true
str.endsWith("lo"); // true
str.repeat(3); // HelloHelloHello
```

★ Template Literals (ES6)

Template literals allow embedded expressions and multi-line strings.

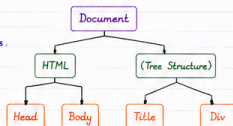
```
let name = "Zara";
let age = 21;
let msg = `My name is ${name} and I am ${age} years old.`;
```

Features :

- Use backticks (` `).
- `\${}` is used to embed expressions.
- Supports multi-line strings.

★ DOM (Document Object Model)

The DOM represents the structure of an HTML document as a tree of objects. JavaScript can access and manipulate the DOM to change content, structure and style.



★ Selecting Elements

- getElementById(id) - Selects one element by id.
- getElementsByClassName(class) - Selects all elements by class.
- getElementsByTagName(tag) - Selects all elements by tag name.
- querySelector(selector) - Selects first element using CSS selector.
- querySelectorAll(selector) - Selects all elements using CSS selector.

```
document.getElementById("myId");
document.getElementsByClassName("box");
document.getElementsByTagName("p");
document.querySelector(".box");
document.querySelectorAll("p");
```

Note :



getE entsBy* methods return HTMLCollection (array-like).
querySelector* methods return single element or NodeList.

JavaScript Complete Notes

7. ES6 Features

ES6 (ECMAScript 2015) introduced many powerful features that make JavaScript easier, cleaner and more efficient.

★ 1. let and const

- let is block scoped and can be updated.
- const is block scoped and cannot be reassigned.

Example:

<pre>let a = 10; a = 20; // allowed console.log(a); // 20</pre>	<pre>const b = 30; b = 40; // Error console.log(b);</pre>
--	--

★ 2. Arrow Functions

- Shorter syntax for writing functions.
- Arrow functions do not have their own this.

Example:

Normal Function	Arrow Function
<pre>function add(a,b) { return a + b; }</pre>	<pre>const add = (a,b) => { return a + b; }; // or even shorter const add = (a,b) => a + b;</pre>

★ 3. Template Literals

- Use backticks (`) instead of quotes.
- Allows embedded expressions using \${}.

Example:

<pre>let name = "Zara"; let age = 21;</pre>	<pre>let msg = `My name is \${name} and I am \${age} years old.`; console.log(msg);</pre>
---	---

★ 4. Destructuring

Extract values from arrays or properties from objects and assign them to variables.

Array Destructuring

```
let fruits = ["Apple", "Banana", "Mango"];
let [a, b, c] = fruits;
// a = Apple, b = Banana, c = Mango
```

Object Destructuring

```
let person = { name: "Zara", age: 21, city: "Delhi" };
let { name, age, city } = person;
// name = Zara, age = 21, city = Delhi
```

★ 5. Spread Operator (...)

Expands an iterable (array, object, etc.) into individual elements.

With Arrays

```
let arr1 = [1, 2, 3];
let arr2 = [4, 5, 6];
let arr3 = [...arr1, ...arr2];
// arr3 = [1, 2, 3, 4, 5, 6]
```

With Objects

```
let obj1 = { a: 1, b: 2 };
let obj2 = { c: 3, d: 4 };
let obj3 = { ...obj1, ...obj2 };
// obj3 = { a: 1, b: 2, c: 3, d: 4 }
```

Note:

- Spread creates a shallow copy.
- Useful for cloning arrays/objects and merging multiple items.

★ 6. Rest Operator (...)

Collects multiple elements into a single array.

In Function Parameters

```
function sum(...numbers) {
  let total = 0;
  for (let n of numbers) total += n;
  return total;
}
sum(1, 2, 3, 4, 5); // 15
```

In Arrays

```
let [a, b, ...rest] = [10, 20, 30, 40, 50];
// a = 10, b = 20, rest = [30, 40, 50]
```

Note:

Rest must be last element in array destructuring and last parameter in function.

★ 7. Default Parameters

Provides default values for function parameters when no value or undefined is passed.

```
function greet(name = "Guest") {
  console.log("Hello, " + name);
}
```

→ greet(); // Hello, Guest
→ greet("Zara"); // Hello, Zara
→ greet(undefined); // Hello, Guest

★ 8. Modules

Allow us to split code into separate files and export/import what is needed.

Export (in math.js)

```
export const pi = 3.14;
export function add(a, b) {
  return a + b;
}
```

Import (in app.js)

```
import { pi, add } from "./math.js";
console.log(pi); // 3.14
console.log(add(2, 3)); // 5
```

Summary

let / const | Arrow Functions | Template Literals | Destructuring | Spread Operator | Rest Operator | Default Parameters | Modules → These features make JS modern, powerful and easy to write.

JavaScript Complete Notes

8. Asynchronous JavaScript

Asynchronous JavaScript allows us to execute tasks in the background without blocking the execution of other code.

① Synchronous vs Asynchronous

Synchronous	Asynchronous
- Tasks are executed one after another. - Each operation waits for the previous operation to complete. - Can block the code execution. - Examples: <code>readFileSync()</code>, <code>simple code</code>.	- Tasks can start, run in the background and do not wait. - Does not block the code execution. - More efficient for I/O operations. - Examples: <code>setTimeout()</code>, <code>Promises</code>, <code>fetch()</code>, <code>async/await</code>.

③ Callback Hell

When multiple callbacks are nested inside each other, the code becomes difficult to read and maintain. This is called "Callback Hell".

```
setTimeout(() => {
  console.log("Step 1");
  setTimeout(() => {
    console.log("Step 2");
    setTimeout(() => {
      console.log("Step 3");
      setTimeout(() => {
        console.log("Step 4");
      }, 1000);
    }, 1000);
  }, 1000);
}, 1000);
```

Problems:

- Hard to read.
- Hard to debug.
- Hard to maintain.
- Leads to more errors.

④ Promises

A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

```
let promise = new Promise((resolve, reject) => {
  let success = true;
  if (success) {
    resolve("Operation Successful!");
  } else {
    reject("Operation Failed!");
  }
});
```

Using `.then()` and `.catch()`

```
promise
  .then((message) => {
    console.log(message); // Operation Successful!
  })
  .catch((error) => {
    console.log(error); // Operation Failed!
  });
```

⑥ async / await (ES8)

- `async/await` is a modern way to work with promises.
- `async` makes a function return a promise.
- `await` makes JavaScript wait for a promise to resolve.

```
async function fetchData() {
  try {
    let response = await fetch("https://api.example.com/data");
    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Error:", error);
  }
}

fetchData();
```

Flow:

- ① `async` function starts.
- ② `await` pauses execution until the promise resolves.
- ③ After resolution, the rest of the code runs.

⑨ Error Handling in Asynchronous Code

Using `.catch()` with Promises

```
fetch("https://api.example.com/data")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log("Error:", err));
```

Using `try...catch` with `async/await`

```
async function loadData() {
  try {
    let res = await fetch("https://api.example.com/data");
    let data = await res.json();
    console.log(data);
  } catch (err) {
    console.log("Error:", err);
  }
}
```

② Callback Functions

A callback is a function passed as an argument to another function and executed after the task is completed.

```
function greet(name, callback) {
  console.log("Hello, " + name);
  callback();
}

function sayBye() {
  console.log("Goodbye!");
}

greet("Zara", sayBye);
// Output: Hello, Zara
           Goodbye!
```

⑤ Promise States

- **pending** - Initial state, neither fulfilled nor rejected.
- **fulfilled** - Operation completed successfully (resolve).
- **rejected** - Operation failed (reject).

Note:

A promise can be either fulfilled or rejected once, but not both.



⑦ fetch() API

Used to make HTTP requests to servers.

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log("Error:", error));
```

⑧ JSON (JavaScript Object Notation)

JSON is a lightweight data format used for exchanging data between a server and a web application.

Example JSON:

```
{
  "id": 1,
  "name": "John",
  "age": 25,
  "isStudent": false
}
```

Convert JS Object to JSON:

`JSON.stringify(obj);`

Convert JSON to JS Object:

`JSON.parse(jsonString);`

Best Practices

- Always handle errors.
- Provide meaningful error messages.
- Show user-friendly messages.
- Use finally if you need to run code regardless of success or failure.



JavaScript Complete Notes

9. Events in JavaScript

Events are actions or occurrences that happen in the system you are programming. JavaScript can "listen" for these events and "react" when they occur.

★ Common Events

Category	Event	Description	Example Use
Mouse Events	click, dblclick, mouseover, mouseout, mousedown, mouseup	Triggered by mouse actions.	Buttons, menus, tooltips.
Keyboard Events	keydown, keypress,keyup	Triggered by keyboard actions.	Form validation, shortcuts.
Form Events	submit, change, focus, blur, input	Triggered by form interactions.	Validation, user input.
Window Events	load, resize, scroll, unload	Triggered by window/browser actions.	Responsive layout, lazy loading.
Document Events	DOMContentLoaded	Triggered when DOM is ready.	Initialize scripts after DOM.

Event Handling

- Using HTML Attribute (Not Recommended)

```
<button onclick="alert('Hello!')">Click Me</button> // Inline event handling
```

- Using addEventListener() (Recommended)

```
const btn = document.getElementById('myBtn');
btn.addEventListener('click', function () { // Attach event listener
  console.log('Button Clicked!');
});
```

Event Object

When an event occurs, an event object is created.

- It contains information about the event.
- Passed to the event handler automatically.

Example :

```
btn.addEventListener('click', function (e) {
  console.log(e.type); // click
  console.log(e.target); // button element
});
```

10. Classes in JavaScript (ES6)

Classes provide a cleaner and more structured way to create objects and handle inheritance.

Class Declaration

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    console.log(`Hello, I am ${this.name}`);
  }
}
```

```
const p1 = new Person("Zara", 21);
p1.greet(); // Hello, I am Zara
```

Inheritance

```
class Student extends Person {
  constructor(name, age, course) {
    super(name, age); // call parent constructor
    this.course = course;
  }
  study() {
    console.log(`${this.name} is studying ${this.course}`);
  }
}
```

```
const s1 = new Student("Ali", 20, "Computer Science");
s1.greet(); // Hello, I am Ali
s1.study(); // Ali is studying Computer Science
```

Key Points :

- constructor() is a special method called automatically when an object is created.
- super() calls the constructor (or methods) of the parent class.
- Classes are not hoisted.

- Everything inside a class is strict mode by default.
- Methods are non-enumerable.



11. try...catch...finally

Used to handle runtime errors and ensure that important code still runs.

```
try {
  let result = 10 / 0;
  console.log(result);
  let x = y; // ReferenceError
} catch (error) {
  console.log("Error:", error.message);
} finally {
  console.log("This always executes.");
}
```

When to Use :

- Handling expected runtime errors.
- Preventing the app from crashing.
- Logging errors.
- Cleaning up resources in finally.



Built-in Error Types

- SyntaxError - Invalid syntax
- ReferenceError - Using undeclared variable
- TypeError - Wrong type of value
- RangeError - Value out of range
- CustomError - User defined errors



Summary : Events make web pages interactive | Classes help create blueprints for objects | Error handling keeps your code reliable and user-friendly.



JavaScript Complete Notes

12. Array Methods (Part - 1)

Arrays in JavaScript come with many built-in methods that make it easy to perform common tasks.

Method	Description	Syntax	Example	Output
<code>push()</code>	Adds one or more elements at the end.	<code>arr.push(item1, item2, ...)</code>	<code>let arr = [1, 2]; arr.push(3, 4);</code>	[1, 2, 3, 4]
<code>pop()</code>	Removes the last element and returns it.	<code>arr.pop()</code>	<code>let arr = [1, 2, 3]; arr.pop();</code>	[1, 2]
<code>shift()</code>	Removes the first element and returns it.	<code>arr.shift()</code>	<code>let arr = [1, 2, 3]; arr.shift();</code>	[2, 3]
<code>unshift()</code>	Adds one or more elements at the beginning.	<code>arr.unshift(item1, item2, ...)</code>	<code>let arr = [2, 3]; arr.unshift(1);</code>	[1, 2, 3]
<code>splice()</code>	Adds, removes or replaces elements.	<code>arr.splice(start, deleteCount, item1, item2, ...)</code>	<code>let arr = [1, 2, 3, 4]; arr.splice(1, 2, 'a', 'b');</code>	[1, 'a', 'b', 4]
<code>slice()</code>	Returns a shallow copy of a portion of an array.	<code>arr.slice(start, end)</code>	<code>let arr = [1, 2, 3, 4]; arr.slice(1, 3);</code>	[2, 3]
<code>concat()</code>	Merges two or more arrays and returns a new array.	<code>arr.concat(arr2, arr3, ...)</code>	<code>let a = [1, 2]; let b = [3, 4]; a.concat(b);</code>	[1, 2, 3, 4]
<code>indexOf()</code>	Returns first index of the element or -1.	<code>arr.indexOf(value)</code>	<code>let arr = [10, 20, 30]; arr.indexOf(20);</code>	1
<code>lastIndexOf()</code>	Returns last index of the element or -1.	<code>arr.lastIndexOf(value)</code>	<code>let arr = [1, 2, 1, 3]; arr.lastIndexOf(1);</code>	2
<code>includes()</code>	Checks if array contains an element.	<code>arr.includes(value)</code>	<code>let arr = [1, 2, 3]; arr.includes(2);</code>	true

13. Higher Order Functions

A function that either takes one or more functions as arguments, or returns a function.

Examples:

- `map()` - creates a new array by applying a function to each element.

```
let numbers = [1, 2, 3];
let squares = numbers.map(function(n) {
  return n * n;
}); // [1, 4, 9]
```

- `filter()` - creates a new array with elements that pass the test.

```
let nums = [1, 2, 3, 4, 5];
let even = nums.filter(function(n) {
  return n % 2 === 0;
}); // [2, 4]
```

- `reduce()` - reduces array to a single value.

```
let nums = [1, 2, 3, 4];
let sum = nums.reduce(function(acc, curr) {
  return acc + curr;
}, 0); // 10
```

Why use them?

- Cleaner code
- Less looping
- Easy to chain
- Functional programming support



- `forEach()` - executes a function for each element.

```
let arr = [10, 20, 30];
arr.forEach(function(item, index) {
  console.log(index + ':' + item);
});
// 0:10
// 1:20
// 2:30
```

- `find()` - returns the first element that matches condition.

```
let users = [
  {id: 1, name: 'Zara'},
  {id: 2, name: 'Ali'},
  {id: 3, name: 'Sara'}
];
let user = users.find(function(u) {
  return u.id === 2;
}); // {id: 2, name: 'Ali'}
```

14. Local Storage & Session Storage

Web Storage allows us to store data in the browser.

Local Storage

- Data has no expiration time.
- Survives browser restart and even system restart.
- Shared across all tabs of the same origin.

```
// Set Item
localStorage.setItem('name', 'Zara');
// Get Item
localStorage.getItem('name'); // 'Zara'
// Remove Item
localStorage.removeItem('name');
// Clear All
localStorage.clear();
```

Key Notes:

- Both store data as key-value pairs of strings.
- Storage limit is around 5-10 MB per origin.



Session Storage

- Data is cleared when the page session ends.
- Survives page refresh but not tab/window close.
- Not shared between different tabs.

```
// Set Item
sessionStorage.setItem('token', 'abcd123');
// Get Item
sessionStorage.getItem('token'); // 'abcd123'
// Remove Item
sessionStorage.removeItem('token');
// Clear All
sessionStorage.clear();
```



Summary: Arrays and higher order functions make data handling easy | Storage helps in preserving data across sessions | Mastering these concepts improves efficiency and code quality.